

Instruction Mix Analyzer

By Ranis Hamidullin

A tool for analyzing RISC-V instruction distributions

Why is This Useful?

1. To understand what types of instructions programs actually use.
2. Hardware designers use this to optimize CPU resources.
3. Compiler writers use this to improve code generation.

Process

1. Read each hex value from the file
2. Extract 7-bit opcode field
3. Map opcode to category (ALU, Memory, Branch, Control Flow, and Other)
4. Count and calculate percentages
5. Print and write the results

Core Logic

```
def opcode_of(inst):  
    return inst & 0x7F # will gets the 7-bit opcode field from a 32-bit instruction word.  
  
def categorize_opcode(op):  
  
    if op == 0x03 or op == 0x23: # Loads and Stores  
        return "Memory"  
  
    if op == 0x63:  
        return "Branches" # Branches (conditional jumps)  
  
    if op == 0x6F or op == 0x67: # JAL and JALR (unconditional jumps and function calls)  
        return "Control Flow"  
  
    if op == 0x33 or op == 0x13: # ALU operations (R-type and I-type arithmetic/logical instructions)  
        return "ALU"  
  
    return "Other"
```

Small Instruction File (20)

```
00A00513
00B50633
40B50633
00C12023
00412283
0062A023
00208663
0000006F
00008067
000002B7
00000317
003100B3
00110113
FE010113
10000093
00500113
00700193
00310233
0040A023
0080066F
```

Instruction Mix Analysis Report

Total valid instructions: 20

Counts and percentages:

ALU	:	10	(50.00%)
Memory	:	4	(20.00%)
Branches	:	1	(5.00%)
Control Flow:		3	(15.00%)
Other	:	2	(10.00%)

Large Instruction File (100)

00A00513
00B50633
40B50633
00C12023
00412283
0062A023
00208663
0000006F

•
•
•

00110113
FE010113
10000093
00500113
00700193
00310233
0040A023
0080066F

Instruction Mix Analysis Report

Total valid instructions: 100

Counts and percentages:

ALU	:	50	(50.00%)
Memory	:	20	(20.00%)
Branches	:	5	(5.00%)
Control Flow:		15	(15.00%)
Other	:	10	(10.00%)

Conclusion (Most to Least Common)

1. ALU operations (usually 45-50% of instructions)
2. Memory operations (usually 25-30%)
3. Branches (usually 10-15%)
4. Jumps/calls (usually 5%)